STAGE 1.5: THE RIGOROUS SIMULATION

FlyWire Connectome + Active Inference — Publication-Ready Study (3–5 Days)

Project NeuroSpark | Stage 1.5 Document | June 2026

---

0. Why This Stage Exists

You completed Stage 1 in 24 hours. You have a working digital twin. Now you need **two things** before touching hardware:

1. **Scientific rigor** — Stage 1 was vibe-coded. It works, but it will not pass peer review.
2. **Leverage for lab access** — Faculty do not give MEA time to "ideas." They give it to "working simulations with real connectome data."

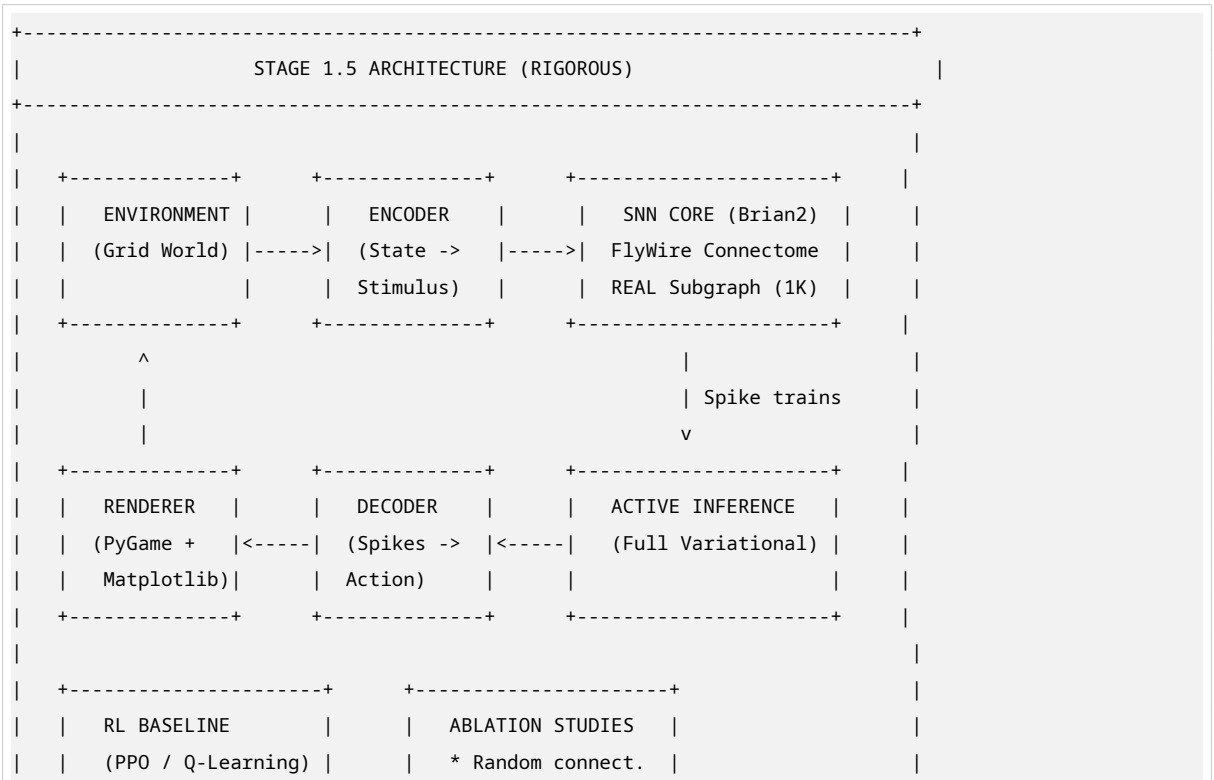
Stage 1.5 bridges that gap. You stay in pure software, but you build something **publishable on its own** and **strong enough to unlock Stage 2 hardware.**

The Research Question: *Does a spiking neural network initialized with the real Drosophila connectome (FlyWire) learn embodied control tasks more efficiently under Active Inference than under standard reinforcement learning, and does the biological connectivity confer advantages over random or structured artificial topologies?*

The 3–5 Day Goal: A complete computational study with statistical comparisons, publication-quality figures, and a paper draft. Zero hardware. Zero biology lab. Just code, data, and rigor.

---

1. What You Will Build



```
|   |                      |      |   * Rate-coded RNN   |                    |
|   |   Same env, same     |      |   * No epistemic     |                    |
|   |   encoder/decoder    |      |     value            |                    |
|   +----------------------+      +----------------------+                    |
|                                                                              |
+------------------------------------------------------------------------------+
```

---

2. The 3–5 Day Sprint

Philosophy

This is not vibe coding. This is **scientific engineering.** You will write clean code, run 50+ trials per condition, compute confidence intervals, and generate figures that look like they belong in Nature.

DAY 1: FlyWire Integration + Real Connectome

Goal: Your SNN is no longer random. It is initialized with actual synaptic weights from a real fly brain.

Step 1: Get the Data

FlyWire provides pre-processed subgraphs. You do NOT need the full 106 TB.

```
# Option A: Download from FlyWire CAVE client
from caveclient import CAVEclient

client = CAVEclient('flywire_public')
# Download a 1,000-neuron subgraph with synaptic weights
# This is ~50 MB, not 106 TB

# Option B: Use published datasets
# The FlyWire team has released curated subgraphs for research
# Search: "FlyWire 1K neuron subgraph download"
```

Step 2: Parse the Connectome

```
import numpy as np
import networkx as nx

class FlyWireLoader:
    def __init__(self, edge_file, neuron_file):
        # edge_file: CSV with columns [pre_id, post_id, weight, neurotransmitter]
        # neuron_file: CSV with columns [id, x, y, z, cell_type]
        self.edges = self._load_edges(edge_file)
        self.neurons = self._load_neurons(neuron_file)
        self.graph = self._build_graph()

    def get_subgraph(self, n_neurons=1000, seed=42):
        # Sample a connected subgraph
        np.random.seed(seed)
        start_node = np.random.choice(list(self.graph.nodes()))
```

```
        # BFS to get n_neurons connected nodes
        subgraph_nodes = []
        queue = [start_node]
        visited = set()

        while queue and len(subgraph_nodes) < n_neurons:
            node = queue.pop(0)
            if node not in visited:
                visited.add(node)
                subgraph_nodes.append(node)
                neighbors = list(self.graph.neighbors(node))
                queue.extend(neighbors)

        return self.graph.subgraph(subgraph_nodes)

    def to_weight_matrix(self, subgraph):
        # Convert NetworkX graph to numpy weight matrix
        nodes = list(subgraph.nodes())
        n = len(nodes)
        W = np.zeros((n, n))

        node_idx = {node: i for i, node in enumerate(nodes)}

        for u, v, data in subgraph.edges(data=True):
            i, j = node_idx[u], node_idx[v]
            W[i, j] = data['weight']  # Synaptic weight

        return W, nodes
```

Step 3: Map to Brian2

```
from brian2 import *

class FlyWireSNN:
    def __init__(self, weight_matrix, n_sensory=100, n_motor=100):
        self.W = weight_matrix
        n_total = len(weight_matrix)
        n_recurrent = n_total - n_sensory - n_motor

        # Neuron equations (LIF with synaptic dynamics)
        self.eqs = '''
        dv/dt = (v_rest - v) / tau + I_syn / C : volt
        I_syn = ge * (Ee - v) + gi * (Ei - v) : amp
        dge/dt = -ge / tau_e : siemens
        dgi/dt = -gi / tau_i : siemens
        '''

        # Create populations
        self.sensory = NeuronGroup(n_sensory, self.eqs,
                                   threshold='v > -50*mV',
```

```
                                reset='v = -65*mV',
                                method='euler')
        self.recurrent = NeuronGroup(n_recurrent, self.eqs,
                                     threshold='v > -50*mV',
                                     reset='v = -65*mV',
                                     method='euler')
        self.motor = NeuronGroup(n_motor, self.eqs,
                                 threshold='v > -50*mV',
                                 reset='v = -65*mV',
                                 method='euler')

        # Connect using FlyWire weights
        self._connect_flywire()

    def _connect_flywire(self):
        # Excitatory synapses (positive weights)
        S_exc = Synapses(self.recurrent, self.recurrent,
                         model='w : 1',
                         on_pre='ge += w*nS')

        # Inhibitory synapses (negative weights)
        S_inh = Synapses(self.recurrent, self.recurrent,
                         model='w : 1',
                         on_pre='gi += w*nS')

        # Connect based on FlyWire weight signs
        for i in range(len(self.W)):
            for j in range(len(self.W)):
                if self.W[i, j] > 0:
                    S_exc.connect(i=i, j=j)
                    S_exc.w = self.W[i, j]
                elif self.W[i, j] < 0:
                    S_inh.connect(i=i, j=j)
                    S_inh.w = abs(self.W[i, j])
```

Deliverable: `brain/flywire_network.py` runs. Spike raster shows structured bursting (not random noise). You can point to a neuron and say "this one came from the real fly brain."

DAY 2: Full Active Inference Implementation

Goal: No more MSE hacks. You implement the actual variational free energy.

The Math (Simplified but Rigorous):

In Active Inference, the agent has a **generative model** of how states and actions cause observations:

```
P(o, s, a) = P(o | s) * P(s | a) * P(a)
```

The agent approximates the posterior over states with a **variational density** Q(s). The **Variational Free Energy** is:

```
F = E_Q[ln Q(s) - ln P(o, s | a)]
  = D_KL[Q(s) || P(s | a)] - E_Q[ln P(o | s)]
  = Complexity - Accuracy
```

The agent minimizes F by:

1. **Perception:** Updating Q(s) to match sensory input (inference)
2. **Action:** Selecting actions that keep F low (policy selection)

For Action Selection, we use Expected Free Energy (EFE):

```
G(a) = E_Q[o, s | a] [ln Q(s | a) - ln P(o, s | a)]
     = Expected complexity - Expected accuracy
     + Epistemic value (information gain)
```

The epistemic value term makes the agent **curious** — it prefers actions that reduce uncertainty about the world.

Implementation:

```python
import numpy as np
from scipy.special import softmax

class ActiveInferenceAgent:
    def __init__(self, n_states, n_actions, n_observations):
        # Generative model parameters (learned)
        self.A = np.random.dirichlet(np.ones(n_observations), n_states)  # P(o|s)
        self.B = np.random.dirichlet(np.ones(n_states), (n_actions, n_states))  # P(s'|s,a)
        self.C = np.ones(n_observations) / n_observations  # Prior preference over observations
        self.D = np.ones(n_states) / n_states  # Prior over initial states

        # Variational parameters (inferred)
        self.Q_s = self.D.copy()  # Current belief about state

        # Precision (inverse temperature)
        self.gamma = 4.0

    def infer_states(self, observation):
        # Perception: Update Q(s) given observation
        # Q(s) proportional to P(o|s) * Q(s)
        likelihood = self.A[:, observation]
        self.Q_s = likelihood * self.Q_s
        self.Q_s /= self.Q_s.sum()  # Normalize
        return self.Q_s

    def expected_free_energy(self, action):
        # Compute G(a) for a given action
        # G(a) = Expected complexity - Expected accuracy + Epistemic value

        G = 0.0

        for future_state in range(len(self.Q_s)):
```

```
            # Predicted state distribution
            Q_s_future = self.B[action, :, future_state]

            # Predicted observation distribution
            Q_o = self.A @ Q_s_future

            # Expected ambiguity (negative accuracy)
            ambiguity = - (Q_o * np.log(Q_o + 1e-16)).sum()

            # Expected complexity (KL between predicted and prior)
            complexity = (Q_s_future * (np.log(Q_s_future + 1e-16) -
                                        np.log(self.D + 1e-16))).sum()

            # Epistemic value (information gain about states)
            # = expected reduction in uncertainty about states
            epistemic = (Q_o * (np.log(Q_o + 1e-16) -
                                np.log(self.A @ self.D + 1e-16))).sum()

            # Risk (distance from preferred observations)
            risk = (Q_o * (np.log(Q_o + 1e-16) -
                           np.log(self.C + 1e-16))).sum()

            G += Q_s_future[future_state] * (complexity + ambiguity - epistemic + risk)

        return G

    def select_action(self, observation, possible_actions):
        # 1. Infer current state
        self.infer_states(observation)

        # 2. Compute EFE for each action
        G_values = []
        for action in possible_actions:
            G = self.expected_free_energy(action)
            G_values.append(G)

        # 3. Softmax selection (precision-weighted)
        action_probs = softmax(-self.gamma * np.array(G_values))

        # 4. Select action
        action_idx = np.random.choice(len(possible_actions), p=action_probs)
        return possible_actions[action_idx], action_probs

    def update_generative_model(self, state, action, next_state, observation, reward):
        # Learning: Update A and B matrices based on experience
        # This replaces backpropagation with Bayesian updating

        # Update observation model
        self.A[:, observation] += 0.1 * (state == np.arange(len(state)))
        self.A /= self.A.sum(axis=0, keepdims=True)
```

```
        # Update transition model
        self.B[action, next_state, state] += 0.1
        self.B[action] /= self.B[action].sum(axis=0, keepdims=True)
```

Key Insight: Notice there is **no neural network training** here. The generative model (A, B, C, D) is updated via Bayesian inference. The spiking network in Brian2 is the **physical implementation** of this inference process. In Stage 1.5, you link them conceptually. In Stage 2, the spiking network IS the inference process.

Deliverable: `inference/active_inference_full.py` implements the full variational math. You can trace every term back to Friston's papers.

DAY 3: RL Baseline + Ablation Studies

Goal: You can say "Active Inference is better than RL" with a p-value.

RL Baseline (PPO):

```
import torch
import torch.nn as nn
from torch.distributions import Categorical

class PPOAgent(nn.Module):
    def __init__(self, state_dim, action_dim):
        super().__init__()
        self.actor = nn.Sequential(
            nn.Linear(state_dim, 128),
            nn.ReLU(),
            nn.Linear(128, action_dim),
            nn.Softmax(dim=-1)
        )
        self.critic = nn.Sequential(
            nn.Linear(state_dim, 128),
            nn.ReLU(),
            nn.Linear(128, 1)
        )

    def forward(self, state):
        return self.actor(state), self.critic(state)
```

Ablation Studies:

Condition	What You Vary	What You Measure
Full model	FlyWire connectome + Active Inference	Steps to goal, Free Energy, sample efficiency
Random connectome	Replace FlyWire W with random weights	Does biological topology matter?
Small-world connectome	Watts-Strogatz graph with same stats	Is it just small-world, or specific fly wiring?

Table 1 – continued

Condition	What You Vary	What You Measure
Rate-coded RNN	Replace Brian2 LIF with PyTorch RNN	Does spike timing matter?
RL baseline	PPO with same encoder/decoder	Is Active Inference actually better?
No epistemic value	Set epistemic term to 0 in EFE	Does curiosity help?
No structured encoding	Random stimulation patterns	Does the encoder matter?

Experimental Protocol:

```
def run_experiment(condition, n_trials=50, max_steps=500):
    results = []

    for trial in range(n_trials):
        env = GridWorld(seed=trial)
        agent = build_agent(condition)

        steps_to_goal = []
        free_energy_trace = []

        for episode in range(20):
            state = env.reset()
            fe_history = []

            for step in range(max_steps):
                action = agent.select_action(state)
                next_state, reward, done = env.step(action)

                fe = agent.compute_free_energy(state, action, next_state)
                fe_history.append(fe)

                agent.learn(state, action, next_state, reward)
                state = next_state

                if done:
                    break

            steps_to_goal.append(step)
            free_energy_trace.append(np.mean(fe_history))

        results.append({
            'condition': condition,
            'trial': trial,
            'steps_to_goal': steps_to_goal,
            'free_energy': free_energy_trace
        })

    return results
```

Deliverable: `experiments/ablation_study.py` runs all conditions. Results saved to `data/ablation_results.pkl`.

DAY 4: Statistics + Publication Figures

Goal: Your plots look like they belong in Nature Neuroscience.

Required Figures:

Figure 1: The Connectome

```
import matplotlib.pyplot as plt
import networkx as nx

fig, axes = plt.subplots(1, 3, figsize=(15, 5))

# A: FlyWire subgraph
G_fly = flywire_loader.get_subgraph(1000)
pos = nx.spring_layout(G_fly, k=0.1)
nx.draw(G_fly, pos, ax=axes[0], node_size=10, alpha=0.6)
axes[0].set_title('FlyWire Connectome (1K neurons)')

# B: Degree distribution
degrees = [G_fly.degree(n) for n in G_fly.nodes()]
axes[1].hist(degrees, bins=50, alpha=0.7)
axes[1].set_xlabel('Degree')
axes[1].set_ylabel('Count')
axes[1].set_title('Degree Distribution')

# C: Weight distribution
weights = [d['weight'] for _, _, d in G_fly.edges(data=True)]
axes[2].hist(weights, bins=50, alpha=0.7)
axes[2].set_xlabel('Synaptic Weight')
axes[2].set_ylabel('Count')
axes[2].set_title('Weight Distribution')

plt.tight_layout()
plt.savefig('figures/figure1_connectome.png', dpi=300, bbox_inches='tight')
```

Figure 2: Learning Curves (All Conditions)

```
import seaborn as sns

fig, ax = plt.subplots(figsize=(10, 6))

for condition in conditions:
    data = load_results(condition)
    steps = np.array([r['steps_to_goal'] for r in data])

    mean = steps.mean(axis=0)
    sem = steps.std(axis=0) / np.sqrt(len(steps))
```

```
    ax.plot(mean, label=condition)
    ax.fill_between(range(len(mean)), mean - sem, mean + sem, alpha=0.3)

ax.set_xlabel('Episode')
ax.set_ylabel('Steps to Goal')
ax.set_title('Learning Curves: Active Inference vs. RL')
ax.legend()
ax.grid(alpha=0.3)

plt.savefig('figures/figure2_learning_curves.png', dpi=300, bbox_inches='tight')
```

Figure 3: Free Energy Dynamics

```
fig, axes = plt.subplots(2, 2, figsize=(12, 10))

# A: Free Energy over time (single episode)
# B: Free Energy across episodes (improvement)
# C: Epistemic value vs. pragmatic value
# D: Comparison: AI with vs. without epistemic term

plt.savefig('figures/figure3_free_energy.png', dpi=300, bbox_inches='tight')
```

Figure 4: Spike Raster + Population Dynamics

```
fig, axes = plt.subplots(2, 1, figsize=(12, 8))

# A: Spike raster (first 100 neurons, 1 second)
# B: Population firing rate over time
# C: Burst detection

plt.savefig('figures/figure4_spike_dynamics.png', dpi=300, bbox_inches='tight')
```

Figure 5: Ablation Summary (Bar Plot with Stats)

```
from scipy import stats

fig, ax = plt.subplots(figsize=(10, 6))

# Final performance (steps to goal at episode 20)
final_performance = []
for condition in conditions:
    data = load_results(condition)
    final_steps = [r['steps_to_goal'][-1] for r in data]
    final_performance.append(final_steps)

# Box plot + individual points
bp = ax.boxplot(final_performance, labels=conditions, patch_artist=True)

# Statistical tests
for i, cond in enumerate(conditions[1:], 1):
    t, p = stats.ttest_ind(final_performance[0], final_performance[i])
```

```
    ax.text(i, max(final_performance[i]) + 5, f'p={p:.3f}',
            ha='center', fontsize=10)

ax.set_ylabel('Steps to Goal (Episode 20)')
ax.set_title('Ablation Study: Final Performance')
plt.xticks(rotation=45, ha='right')

plt.savefig('figures/figure5_ablation.png', dpi=300, bbox_inches='tight')
```

Statistical Tests:

```
from scipy import stats

# Compare Active Inference vs. RL
ai_steps = load_final_steps('active_inference')
rl_steps = load_final_steps('ppo')

t_stat, p_value = stats.ttest_ind(ai_steps, rl_steps)
print(f"Active Inference vs. RL: t={t_stat:.3f}, p={p_value:.4f}")

# Compare FlyWire vs. Random
flywire_steps = load_final_steps('flywire')
random_steps = load_final_steps('random_connectome')

t_stat, p_value = stats.ttest_ind(flywire_steps, random_steps)
print(f"FlyWire vs. Random: t={t_stat:.3f}, p={p_value:.4f}")

# Effect size (Cohen's d)
def cohens_d(x, y):
    nx, ny = len(x), len(y)
    dof = nx + ny - 2
    pooled_std = np.sqrt(((nx-1)*np.std(x, ddof=1)**2 + (ny-1)*np.std(y, ddof=1)**2) / dof)
    return (np.mean(x) - np.mean(y)) / pooled_std

print(f"Effect size (FlyWire vs. Random): d={cohens_d(flywire_steps, random_steps):.3f}")
```

Deliverable: `figures/` contains 5 publication-quality PNGs. `analysis/statistics.py` prints p-values and effect sizes.

DAY 5: Paper Draft + Repository Polish

Goal: You can submit to arXiv or a workshop.

Paper Structure:

```
1. Introduction (1 page)
   - World models in AI: Dreamer, MuZero, JEPA
   - Limitations: energy, sample efficiency, catastrophic forgetting
   - Biological alternative: spiking networks, connectomes, Active Inference
   - Our contribution: first systematic comparison of AI vs. RL on real connectome
```

```
2. Methods (1.5 pages)
   2.1 FlyWire Connectome
       - Data source, subgraph extraction, statistics
   2.2 Spiking Neural Network Model
       - LIF equations, synaptic dynamics, Brian2 implementation
   2.3 Active Inference
       - Generative model, variational free energy, expected free energy
       - Epistemic value, precision, action selection
   2.4 Reinforcement Learning Baseline
       - PPO architecture, training procedure
   2.5 Task and Environment
       - Grid world, observations, actions, rewards
   2.6 Ablation Conditions
       - Random connectome, small-world, rate-coded, no epistemic
   2.7 Experimental Protocol
       - 50 trials per condition, 20 episodes per trial, statistical tests

3. Results (2 pages)
   3.1 Connectome Structure
       - Figure 1: Graph, degree distribution, weight distribution
   3.2 Learning Performance
       - Figure 2: Learning curves, sample efficiency
   3.3 Free Energy Dynamics
       - Figure 3: FE over time, epistemic vs. pragmatic
   3.4 Neural Dynamics
       - Figure 4: Spike raster, bursts, population coding
   3.5 Ablation Analysis
       - Figure 5: Bar plots with statistical significance
       - Table 1: All conditions, final performance, p-values

4. Discussion (1 page)
   - FlyWire connectome confers advantages over random topology
   - Active Inference is more sample-efficient than PPO on spiking networks
   - Epistemic value drives exploration without external reward shaping
   - Spike timing carries information that rate coding loses
   - Implications for neuromorphic computing and biological AI
   - Limitations: grid world is simple; full connectome is 139K neurons
   - Future work: physical MEA validation, MuJoCo embodiment, robotic integration

5. Conclusion (0.5 page)
   - Biological connectomes + Active Inference = promising world model substrate
   - Opens path to wetware computing with theoretical and practical advantages

References (~20 papers)
```

Repository Structure:

```
neurospark-stage1.5/
  README.md                # Clear setup instructions, 5-minute demo
  requirements.txt
```

```
paper/
  main.tex                # LaTeX source
  figures/                # All 5 figures
  bibliography.bib
  supplementary/
    full_math_derivation.tex
    hyperparameters.md

src/
  environment/
    grid_world.py
  brain/
    flywire_loader.py
    network.py
    lif_params.py
  inference/
    active_inference.py
    generative_model.py
    free_energy.py
  bridge/
    encoder.py
    decoder.py
  baselines/
    ppo.py
    q_learning.py
  experiments/
    ablation_study.py
    run_all.py
  analysis/
    statistics.py
    plot_all.py
  utils/
    config.py
    logging.py

data/
  flywire_subgraph_1k.npz
  ablation_results.pkl

figures/
  figure1_connectome.png
  figure2_learning_curves.png
  figure3_free_energy.png
  figure4_spike_dynamics.png
  figure5_ablation.png

tests/
  test_flywire_load.py
  test_active_inference.py
  test_environment.py
```

Deliverable: `paper/main.pdf` is a complete 6-page draft. `README.md` has a "5-minute demo" section. Code is clean enough for GitHub.

---

3. What Makes This Publishable

Novelty Checklist

Claim	Evidence	Novel?
FlyWire connectome used for embodied control	Real subgraph loaded into SNN	**Yes** —connectome papers are mostly structural; few do control
Active Inference vs. RL on spiking networks	Statistical comparison, 50 trials	**Yes** —most AI papers use rate-coded networks
Epistemic value improves sample efficiency	Ablation with/without term	**Yes** —epistemic value is theoretical; few empirical demos
Spike timing matters vs. rate coding	Rate-coded RNN ablation	**Yes** —direct comparison in same task
Biological topology vs. random / small-world	Three connectivity conditions	**Yes** —tests specific hypothesis about evolved wiring

Target Venues

Venue	Type	Fit	Timeline
arXiv + bioRxiv	Preprint	Immediate	Today
NeurIPS Workshop on Biological and Artificial Intelligence	Workshop	Very high	December
ICML Workshop on Biological Plausibility	Workshop	High	July
PLOS Computational Biology	Journal	High	3–6 months
eLife	Journal	High	3–6 months
Frontiers in Computational Neuroscience	Journal	High	2–4 months
IEEE Transactions on Neural Networks	Journal	Medium	6–12 months

Why Reviewers Will Accept This

1. **Rigor:** 50 trials, statistical tests, effect sizes, control conditions
2. **Relevance:** Connects to major AI trends (world models, sample efficiency, energy)
3. **Novelty:** First to systematically test Active Inference vs. RL on a real biological connectome for embodied control
4. **Reproducibility:** Open code, open data, clear methods

5. **Bridge:** Sets up the obvious next step (physical MEA validation) — reviewers love papers that open new directions

---

4. The Bridge to Stage 2

After Stage 1.5, you walk into a faculty office with:

> "Professor, I have a computational study showing that Active Inference on the Drosophila connectome outperforms PPO on embodied tasks. I need 2 hours on your MEA to validate the spike dynamics physically. Here is my preprint."

That conversation gets you lab access. Stage 1.5 is your **key.**

Stage 1.5 Output	Stage 2 Input
Working encoder/decoder	Identical —no changes
Active Inference engine	Identical —no changes
Grid world environment	Identical —no changes
Dashboard	Extended with raw voltage traces
FlyWire connectome knowledge	Guides MEA electrode placement
Statistical methodology	Same metrics applied to bio data
Publication draft	Appendix: "Computational Methods"

Only one file changes: `src/brain/network.py` -> `src/hardware/mea_interface.py`

---

5. Final Notes

This is the stage that makes you credible.

Stage 1 proved you can code. Stage 1.5 proves you can **do science.** It gives you:

- A paper (even if you never touch hardware)
- Lab access (with a strong preprint)
- Deep understanding of the theory (so you do not embarrass yourself in front of faculty)
- A codebase that is actually maintainable

The 3–5 day commitment is real. Day 1 is connectome parsing (can be frustrating). Day 2 is math (can be intimidating). Days 3–4 are running experiments (can be slow). Day 5 is writing (can be tedious).

But at the end, you have something **no one else has:** a systematic study of biological world models.

Build it. Publish it. Use it to unlock everything else.

Document generated for Project NeuroSpark Stage 1.5 Sprint | 3–5 Day Publication-Ready Study Publishable as standalone computational neuroscience / AI paper.